경력기술서

역할 표기 — **단독** 설계부터 구현 · 운영까지 직접 · **주도** 방향과 핵심 구현을 맡고 일부는 동료와 · **공동** 동료와 함께 설계하고 맡은 부분을 구현 · **참여** 사수가 이끈 작업에서 맡은 영역을 구현

상세 경력기술서
모든 사례의 아키텍처 그림 · 판단 근거 · 역할을 담은 전체 버전
보기 →

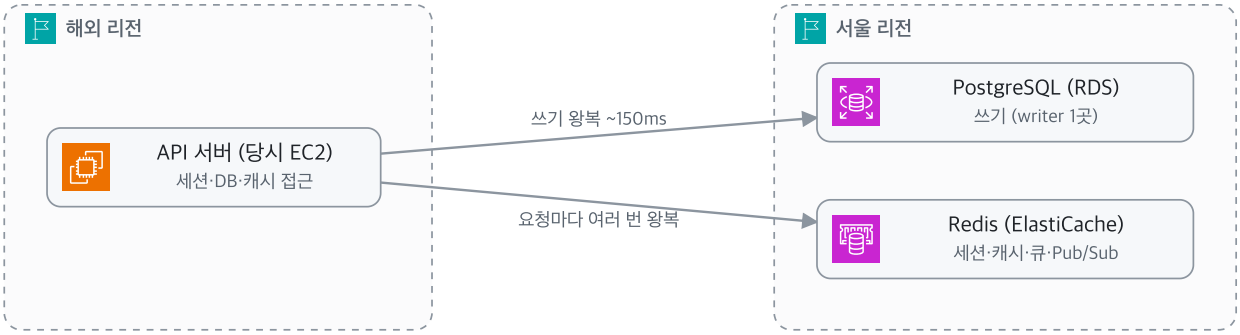
플리토 · 실시간 AI 통번역

글로벌 리전 레이턴시 개선

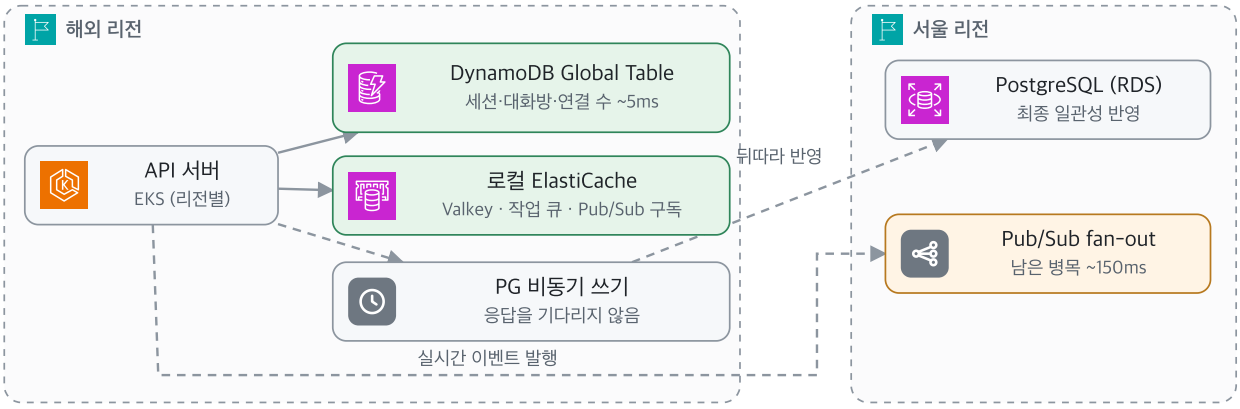
2025.11 – 2026.05 · 단독 · 설계 · 구현

DynamoDB · Redis · PostgreSQL · 재현 실험 코드 →

Before — 모든 쓰기가 서울 리전으로 왕복



After — 리전 로컬 쓰기 + 비동기 동기화



문제 해외 리전에서 쓰면 국내보다 4~5배 느리다는 피드백. 유력한 원인은 서울 리전에 묶인 DB·Redis 쓰기 왕복(1회 약 150ms)이 요청마다 여러 번 발생하는 구조.

판단 Aurora Global Database·Aurora DSQL 검토 — 쓰기는 여전히 한 리전이거나 대륙 간 미지원이고 비용도 더 높아 보류. 실시간성이 강한 데이터만 DynamoDB와 Redis를 활용한 리전 로컬 쓰기로 옮기기로 결정.

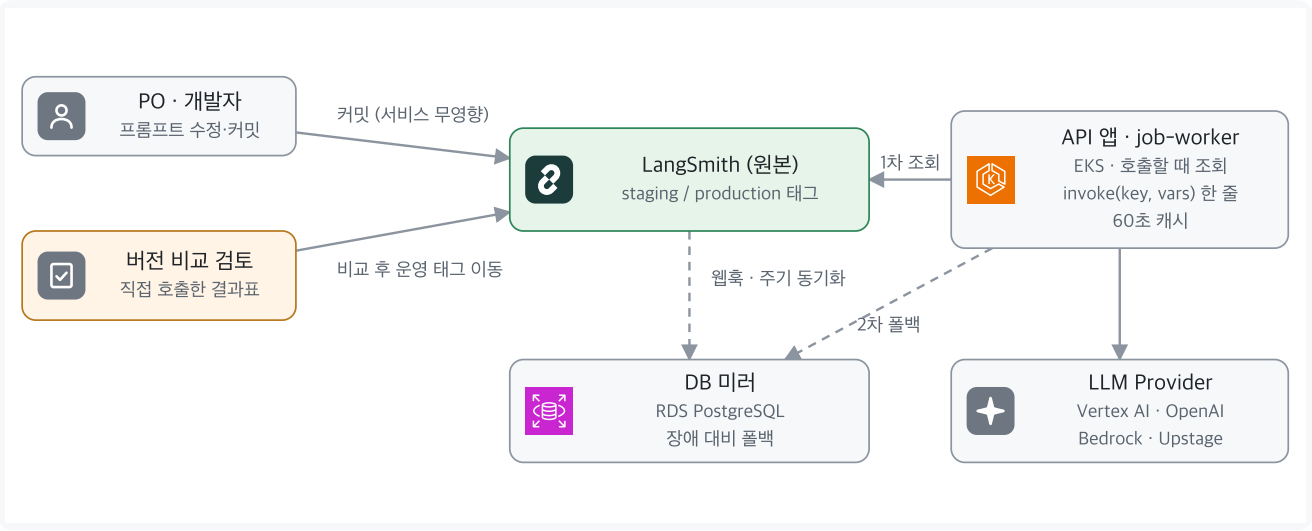
해결 대화방·세션·참가자 연결 수를 DynamoDB Global Tables로, 작업 큐를 리전별 Redis로 옮기고, 실시간 메시지는 모든 리전에 발행해 각 리전에서 구독. PostgreSQL은 응답을 기다리지 않고 뒤따라 쓰게 하고(DynamoDB 실패 시 PostgreSQL로 처리), 설정값 하나로 DynamoDB를 끄면 PostgreSQL에 바로 쓰는 기존 구조로 돌아가게 함.

결과 Close API **1,673 → 327ms (-80%)**, Create API **2,630 → 90ms (-97%)** — 해외 상황을 재현한 개발 환경에서 15회 측정. 남은 병목은 Pub/Sub fan-out.

LLM 프롬프트 관리 체계

2026.06 단독 · 설계 · 구현

LangSmith



문제 프롬프트가 코드 안 문자열이라 문장 하나를 고쳐도 배포가 필요했고, 프롬프트 버전 관리 자체가 없어 고칠 때마다 생긴 버전들 사이의 차이와 품질을 비교할 방법이 없었음.

해결 LangSmith를 원본으로 두어 프롬프트를 커밋 단위로 버전 관리하고, 서버는 호출할 때 직접 조회(60초 캐시, 장애 시 DB 미러 폴백). 개발/운영은 태그로 분리하고, 새 버전은 개발 태그로 먼저 써 본 뒤 에이전트를 활용하여 프롬프트를 직접 호출한 테스트 결과표로 이전 버전과 비교하고 운영 태그를 옮기는 순서를 팀 릴리스 절차로 정함.

결과 코드 배포 없이 프롬프트를 고쳐 약 1분 안에 반영 — PO·PM용 사용 가이드를 함께 제공.

그 밖의 사례

제품 도메인 오너 — 기획부터 개발 분배까지 단독

AX 전환이 관성에 막혀 있어, 도메인 하나를 맡아 PRD · Figma 시안 · 서버/클라 명세를 한 벌로 쓰고 직군별 컨펌 후 개발을 분배 (진행 중)

상세 →

지식그래프 기반 초개인화 단독

사용자의 용어 수정 · AI 지시를 시간 정보가 있는 사용자별 지식그래프로 쌓아, 최신 정보를 우선으로 번역 · AI 답변에 반영 — 상용 반영

상세 →

로그 · 배포 운영 안정화 주도

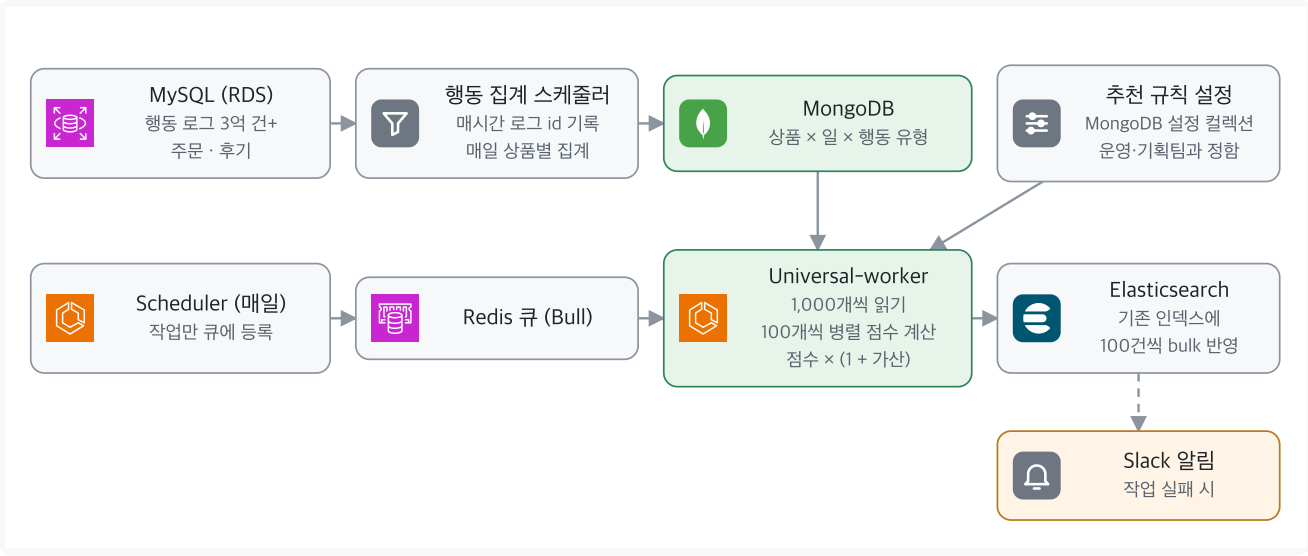
디스크 파일뿐이던 로그를 OpenSearch로 모으고 디스크 알림 · 보관 정책을 갖춤. 시크릿 변경은 Jenkins 파이프라인으로, 배포는 실패 시 자동 롤백

상세 →

추천 상품 스케줄러 재구축

2024.06 – 2024.11 단독 · 설계 · 구현

MySQL · MongoDB · Elasticsearch · Bull



방식 개인화 모델이 아니라, 브랜딩·운영팀과 기획자가 통계 데이터를 보고 정한 규칙(행동별 가중치)으로 상품 점수를 매겨 추천 순서를 내려주는 룰 기반 추천. 트렌드순 · 인기순은 운영 담당자가 정한 유효 범위의 데이터(기간 · 행동 유형) 기준으로 점수를 가산해 순서를 매김.

문제 상품마다 3억 건 이상의 행동 로그에서 2주치 클릭·좋아요·장바구니 수를 세는 서브쿼리를 붙이고 주문·리뷰까지 함께 조회해 실행에 60분 이상 걸림. 가중치(예: 클릭 5 · 좋아요 10 · 장바구니 20)도 쿼리에 고정돼 있었지만, 실행 부하가 커서 3년 넘게 손대지 못함.

해결 3억 건 이상의 원본 로그를 매번 조회하는 대신, 매시간 그 시각의 로그 id를 MongoDB에 기록해 두고 매일 이 id 범위로 상품별 · 행동 유형별 하루 집계를 MongoDB에 저장. 점수 계산은 원본 로그가 아닌 이 집계를 읽어 상품을 1,000개씩 읽고 100개씩 병렬로 계산하도록 다시 작성하고, 스케줄러는 작업만 큐에 넣고 계산은 별도 워커(Universal-worker)가 맡도록 분리. 가중치 · 기간 · 정렬 조건은 쿼리에서 빼 MongoDB 설정 컬렉션으로 옮기고, 코드 배포 없이 백오피스에서 내부 관리자가 직접 값을 조율하며 추천 결과를 볼 수 있게 함. 작업이 실패하면 Slack으로 알림.

판단 전체 상품을 새 인덱스로 만들어 통째로 바꾸는 방식(별칭 교체 · 블루그린 인덱스)은 검색 쿼리가 인덱스 이름을 직접 지정하고 있어 모든 검색 쿼리를 함께 고쳐야 하고, 데이터 교체와 배포 사이에 검색이 잠깐 끊길 위험이 있었음 → 인덱스는 그대로 두고 상품 문서의 추천 순번(트렌드 · 인기)만 100건씩 bulk로 차례로 갱신(충돌 시 재시도).

결과 스케줄러 실행 시간 **60분 → 15분**.

그 밖의 사례

알림 메시지 워커 단독

100만 명 대상 알림톡 발송을 Redis 큐 기반 워커 서버로 분리하고, 실패 건만 재시도 · 이력 보관

상세 →

Elasticsearch 스냅샷 마이그레이션 자동화 단독

스냅샷 생성 → 상태 확인 → 스테이징 복원을 Step Functions · Lambda로 자동화해 주기 실행

상세 →

Feature Flag 기반 Trunk-Based 개발 공동

Remote Config 기반 트렁크 개발을 동료와 설계하고, Swagger에 API별 배포 상태 라벨을 자동으로 붙이는 기능 구현

상세 →

AWS 권한 정책 주도

전체 권한 단일 그룹을 환경 · 리소스별 9개 그룹과 역할 6단계의 최소 권한 구조로 재설계

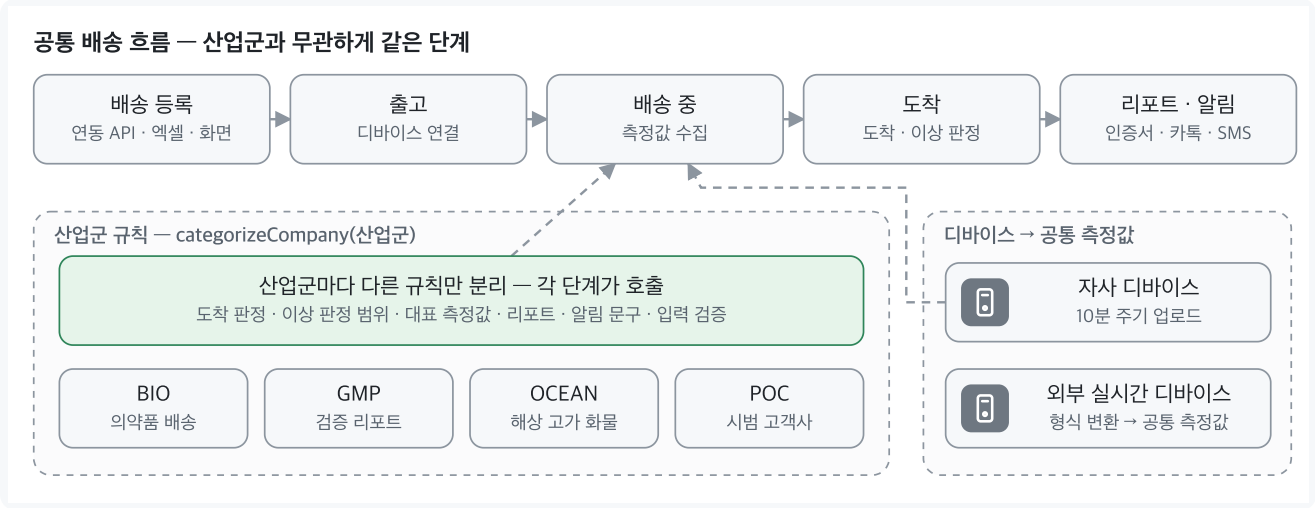
상세 →

윌로그 · 물류 배송 모니터링

공통 배송 흐름 + 산업군 규칙

공동 · 플랫폼 백엔드 대부분 구현 사수가 전 코드 리뷰

의약품 · GMP 검증 · 해상 고가 화물 · 디바이스 호환



문제 의약품 · GMP 검증 · 해상 고가 화물은 도착을 판정하는 기준, 이상으로 보는 온도 · 습도 범위, 리포트 양식, 알림 문구가 서로 다름. 산업군마다 서비스를 새로 만들면 같은 배송 로직이 계속 복제됨.

해결 등록 → 출고 → 배송 중 → 도착 → 리포트 · 알림 흐름은 하나로 두고, 산업군마다 다른 규칙(도착 판정 · 이상 판정 범위 · 대표 측정값 · 리포트 · 알림 문구 · 입력 검증)만 산업군 값 하나로 고르는 규칙 묶음으로 분리. 자사 디바이스와 외부 실시간 디바이스도 공통 측정값 형식으로 바꿔 같은 흐름에 넣음. 예: 도착은 전체 도착 · 부분 도착 · 임의 도착 시간으로, 반품은 부분 · 전체 반품으로 나누고, 이상 판정 범위는 고객사 → 배송그룹 → 배송 순으로 상위 값을 기본값으로 물려받게 함.

결과 MVP 이후 배차 관제, 제약사 GMP 검증 플랫폼, 해상 고가 화물 플랫폼을 같은 코드베이스에 차례로 출시 (2023.03 – 2023.08).

그 밖의 사례

기능 기준 서버 분리 참여

화면 API · 디바이스 · 고객사 연동 · 알림 서버 분리 중 알림 · 고객사 연동 Lambda 구현

상세 →

레거시 무중단 이관 · 고객사 DB 연동 참여

배송 200만 건 · 시계열 5억 건 무중단 이관 검증, 고객사별 DB 연동 Lambda 구현

상세 →

실시간 측정값 · 조회 성능 참여

Redis Pub/Sub + GraphQL Subscription으로 측정값을 새로고침 없이 전달, 도착 배송 캐시로 조회 3초 → 1초

상세 →

문제 해결 · 협업 단독

고객사 연동 API 지연 20초 → 5초, 사내 관리자 서비스(RMS) 백엔드, 현장 VOC로 맞춤 개발

상세 →